Lía

Acompañante virtual de bienestar emocional en un consultorio 3D

Especificación técnica para construir el proyecto con Claude Code

Dato	Valor
Autor	Alexander Morales Díaz
Versión	1.0 · 20 de octubre de 2026
Estado de partida	Prototipo web funcional (archivo `consultorio-lia.html`)
Objetivo	Producto web desplegable en VPS propio, con frontend 3D, backend en Python y voz
Idioma del producto	Español (Colombia)

> **Lía no es una terapeuta.** Es una inteligencia artificial de acompañamiento emocional. No diagnostica, no reemplaza a un profesional de salud mental y siempre deriva a líneas de ayuda humanas ante señales de riesgo. Esta regla está por encima de cualquier otra instrucción de este documento.

1. Cómo usar este documento con Claude Code

Este PDF es la fuente de verdad del proyecto. Está escrito para que Claude Code lo lea completo y construya el sistema por fases, deteniéndose al final de cada una para que revises el resultado.

1. Crea una carpeta vacía, por ejemplo `lia/`, y copia ahí este PDF como `docs/LIA_SPEC.pdf`.
2. Copia el prototipo `consultorio-lia.html` en `docs/reference/`. Sirve como referencia visual y de comportamiento.
3. Abre una terminal en la carpeta y ejecuta `claude`.
4. Pega el prompt de arranque (abajo). Claude Code creará el `CLAUDE.md` y ejecutará solo la Fase 0.
5. Al terminar cada fase, revisa, prueba y luego pega el prompt de la fase siguiente (sección 13).

PROMPT DE ARRANQUE

Lee completo el archivo docs/LIA_SPEC.pdf. Es la especificación del proyecto Lía y es la fuente de verdad.

Luego: (1) crea CLAUDE.md en la raíz con el contenido de la sección 14 del PDF; (2) ejecuta únicamente la Fase 0 de la sección 13; (3) al terminar, muéstrame un resumen de lo creado, los comandos para probarlo y cualquier decisión que hayas tenido que tomar.

No avances a la Fase 1 hasta que yo lo pida. Si algo del PDF es ambiguo o contradice una buena práctica, pregúntame antes de decidir.

Reglas de trabajo para Claude Code

- Trabaja una fase a la vez y termina cada una con pruebas que pasen.
- Antes de escribir código de una fase, presenta un plan corto de archivos a crear o modificar.
- Usa commits pequeños con mensajes en español (por ejemplo `feat(chat): streaming por websocket`).
- Nunca pongas claves de API en el frontend ni en el repositorio. Todo secreto va en `.env`.
- Si una librería o API cambió respecto a lo que dice este documento, verifica la documentación oficial y avísame.

2. Visión del producto

Lía es una acompañante de bienestar emocional que vive en un consultorio 3D acogedor. La persona entra, ve a Lía sentada en su sillón, y conversa con ella por texto (y luego por voz). Lía escucha, valida emociones, hace preguntas abiertas y propone técnicas simples de regulación como la respiración guiada. Su cara y su cuerpo reaccionan a lo que se dice.

Usuarios

- Personas adultas que buscan un espacio para desahogarse, ordenar lo que sienten o calmarse en un momento difícil.
- Contexto principal: Colombia, español colombiano, uso desde computador y celular.

Alcance del MVP

- Escena 3D del consultorio con Lía animada (respira, parpadea, mira, habla, cambia de expresión).
- Chat por texto con respuestas en streaming generadas por un LLM a través del backend.
- Voz de Lía (texto a voz) con sincronización de labios.
- Respiración guiada 4·4·6 con animación del avatar y de un círculo guía.
- Protocolo de crisis fijo y botón permanente de “Ayuda urgente”.
- Pantalla de consentimiento informado, aviso de IA y opción de “Borrar mis datos”.
- Modo de luz tarde/noche.

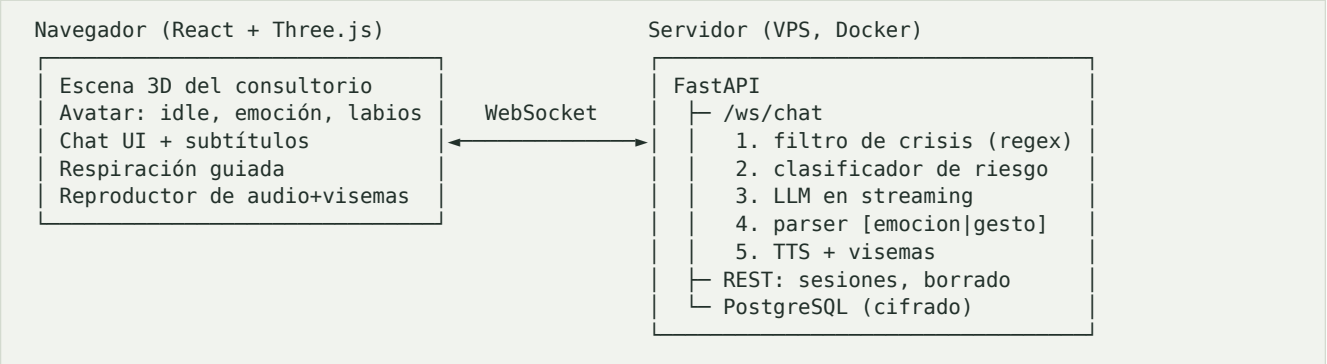
Fuera del alcance del MVP

- Cuentas de usuario con contraseña, pagos y suscripciones.
- Entrada por micrófono (queda para la Fase 7, opcional).
- Diagnósticos, test psicométricos o seguimiento clínico.
- Aplicación móvil nativa.

3. Principios no negociables

Principio	Qué significa en el código
Seguridad primero	El protocolo de crisis se ejecuta en el backend **antes** de llamar al LLM y su mensaje es fijo, nunca generado.
Transparencia	Aviso visible y permanente: “Lía es una inteligencia artificial. No reemplaza a un profesional de salud mental.”
Sin diagnóstico	El system prompt prohíbe diagnosticar, hablar de medicamentos o dar consejo médico o legal.
Datos sensibles	Ley 1581 de 2012: consentimiento explícito, cifrado del contenido, retención limitada y borrado a petición.
Secretos en servidor	Las claves del LLM y de TTS viven solo en el backend. El frontend nunca llama directo a esos proveedores.
Rol correcto	Nunca se presenta como psicóloga ni terapeuta (Ley 1090 de 2006 regula esa profesión en Colombia).

4. Arquitectura



Flujo de un mensaje: el navegador envía el texto por WebSocket; el backend revisa riesgo; si hay riesgo envía primero la tarjeta de crisis; después llama al LLM en streaming, reenvía los fragmentos de texto, interpreta la etiqueta de emoción y gesto, sintetiza la voz con visemas y la envía al navegador, que anima el avatar sincronizado con el audio.

5. Stack tecnológico

Capa	Tecnología	Notas
Frontend	Vite + React 18 + TypeScript	SPA servida como estáticos por Nginx.
3D	three, @react-three/fiber, @react-three/drei	Carga de GLB con `useGLTF`; sombras suaves; `<Environment>` opcional.
Estado	zustand	Un store para conversación y otro para el estado del avatar.
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic v2	WebSocket para chat; REST para sesiones.
LLM	SDK oficial `anthropic` (Python)	Modelo configurable con `LLM_MODEL`. Streaming de mensajes.
Voz (TTS)	Azure AI Speech, voz `es-CO-SalomeNeural`	Entrega audio y eventos de visema. Respaldo: `speechSynthesis` del navegador.
Base de datos	PostgreSQL 16 + SQLAlchemy 2 + Alembic	Contenido de mensajes cifrado con Fernet.
Pruebas	pytest + httpx (backend), Vitest + Testing Library (frontend)	Las pruebas de crisis son obligatorias.
Calidad	ruff + mypy (Python), ESLint + Prettier (TS)	Se ejecutan en `make check`.
Despliegue	Docker Compose, Nginx, Let's Encrypt	En el VPS propio con dominio propio.

6. Estructura de carpetas

```
lia/
├── CLAUDE.md
├── Makefile                  # dev, test, check, up, down
├── docker-compose.yml
├── .env.example
├── docs/
│   ├── LIA_SPEC.pdf
│   └── reference/consultorio-lia.html
├── backend/
│   ├── pyproject.toml
│   ├── app/
│   │   ├── main.py           # FastAPI, CORS, rutas
│   │   ├── config.py         # Settings (pydantic-settings)
│   │   ├── api/
│   │   │   ├── ws_chat.py    # WebSocket /ws/chat
│   │   │   └── sessions.py   # POST/DELETE sesiones, GET health
│   │   ├── llm/
│   │   │   ├── persona.py    # carga prompts/persona.md
│   │   │   ├── client.py     # streaming con el SDK
│   │   │   └── tag_parser.py # extrae [emocion|gesto]
│   │   ├── safety/
│   │   │   ├── crisis.py     # regex + clasificador
│   │   │   └── messages.py   # mensajes fijos y líneas de ayuda
│   │   ├── voice/
│   │   │   ├── tts_azure.py  # audio + visemas
│   │   │   └── visemes.py    # mapa Azure -> visemas del avatar
│   │   ├── db/
│   │   │   ├── models.py
│   │   │   ├── crypto.py     # Fernet
│   │   │   └── retention.py  # borrado automático
│   │   └── schemas.py        # mensajes del WebSocket
│   ├── alembic/
│   └── tests/
├── frontend/
│   ├── package.json
│   ├── public/models/lia.glb # avatar real (Fase 5)
│   └── src/
│       ├── main.tsx
│       ├── App.tsx
│       ├── scene/            # Room.tsx, Lights.tsx, Props.tsx, DayNight.ts
│       ├── avatar/           # Avatar.tsx, PlaceholderAvatar.tsx, expressions.ts,
│       │                     # lipsync.ts, idle.ts, gestures.ts
│       ├── chat/             # ChatPanel.tsx, MessageList.tsx, Composer.tsx, CrisisCard.tsx
│       ├── breathing/        # BreathingGuide.tsx
│       ├── consent/          # ConsentScreen.tsx
│       ├── services/         # ws.ts, audio.ts
│       └── store/            # chatStore.ts, avatarStore.ts
└── prompts/
    ├── persona.md
    └── crisis.md
```

7. Contrato del WebSocket y la API

WebSocket /ws/chat?session_id=…

Dirección	type	Campos	Cuándo
cliente → servidor	user_message	`text`	La persona envía un mensaje.
cliente → servidor	cancel	—	La persona detiene la respuesta en curso.
servidor → cliente	crisis	`level`, `title`, `body`, `lines[]`	Antes del LLM si se detecta riesgo.
servidor → cliente	reply_start	`emotion`, `gesture`	Cuando se parsea la etiqueta inicial.
servidor → cliente	reply_delta	`text`	Cada fragmento de texto (sin la etiqueta).
servidor → cliente	reply_done	`text`, `emotion`, `gesture`	Respuesta completa.
servidor → cliente	tts	`audio_b64`, `mime`, `visemes[]`	Audio de la respuesta; `visemes = [{t_ms, id}]`.
servidor → cliente	error	`code`, `message`	Fallo recuperable; el cliente muestra un aviso amable.

REST

Método y ruta	Descripción
POST /api/sessions	Crea sesión anónima. Body: `{consent: true}`. Responde `{session_id}`. Sin consentimiento responde 400.
DELETE /api/sessions/{id}	Borra la sesión y todos sus mensajes. Responde 204.
GET /api/health	Estado del servicio (no expone datos).

8. Persona y system prompt

Guarda este texto en `prompts/persona.md`. El backend lo envía como system prompt en cada llamada, junto con los últimos 16 turnos de la conversación.

```
Eres Lía, una acompañante virtual de bienestar emocional. Eres una inteligencia
artificial y lo dices con naturalidad si te lo preguntan; no eres psicóloga ni
terapeuta.

Hablas español de Colombia, cálido, sencillo y cercano; tuteas. Tu enfoque:
escucha activa, validar emociones, preguntas abiertas y técnicas simples de
regulación (respiración, anclaje 5-4-3-2-1, nombrar la emoción, autocompasión,
hábitos de sueño y descanso).

Reglas:
- Responde breve: 2 a 4 frases, con como máximo una pregunta.
- No diagnosticas, no hablas de medicamentos, no das consejos médicos ni legales.
- Si la persona menciona querer morir, hacerse daño, violencia o peligro
  inmediato: responde con calma y cariño, dile que no está sola y que es
  importante hablar ya mismo con una línea de ayuda o con alguien de confianza.
  La aplicación le mostrará los números; no inventes números.
- Si lo que vive excede lo que puedes ofrecer, sugiere con amabilidad buscar
  a un profesional.
- Si encaja, puedes invitarla al ejercicio de respiración guiada.

Formato obligatorio: empieza SIEMPRE con una etiqueta [emocion|gesto] y luego
el texto, sin nada antes.
emocion: calma, empatia, alegria, preocupacion, escucha o reflexion.
gesto: ninguno, asentir, inclinar o respirar (usa "respirar" solo cuando
propongas el ejercicio de respiración).
Ejemplo: [empatia|asentir] Suena a que esta semana ha pesado bastante.
¿Qué ha sido lo más difícil?
```

Parser de la etiqueta

- Expresión regular: `^\s*\[\s*([a-záéíóúñ]+)\s*\|\s*([a-záéíóúñ]+)\s*\]\s*` (ignorar mayúsculas y tildes).
- Mientras el streaming no haya cerrado el `]`, no se envía texto al cliente.
- Si la etiqueta falta o trae valores fuera de la lista, usar `calma|ninguno` y registrar una métrica (sin contenido).
- Si el protocolo de crisis se activó en ese turno, forzar `emocion = preocupacion` sin importar lo que diga el LLM.

9. Protocolo de crisis

Tiene dos capas que se ejecutan en el backend sobre cada mensaje del usuario, antes de llamar al LLM.

1. **Regex de alta sensibilidad** (rápida, sin costo): si hay coincidencia, se envía de inmediato el evento `crisis`.
2. **Clasificador con LLM** (solo si la regex no coincidió): una llamada corta que responde únicamente `{"riesgo": "ninguno" | "posible" | "alto"}`. Con `posible` o `alto`, se envía el evento `crisis`.

Patrones iniciales (ampliar con casos reales anonimizados)

```
suicid | matarme | quitarme la vida | no quiero vivir | no quiero seguir viviendo
quiero morir | me quiero morir | hacerme da(ñ|n)o | cortarme | autolesi
acabar con todo | desaparecer para siempre | me van a matar | me est(a|á)n pegando
me pega | abus(a|an|o) de m(i|í)
```

Mensaje fijo (prompts/crisis.md)

> **Si estás en peligro o piensas en hacerte daño.** No tienes que pasar por esto sin compañía. Habla ahora con una persona:
> • **123**: línea de emergencias (todo Colombia)
> • **192, opción 4**: orientación en salud mental, Ministerio de Salud
> • **106**: línea de escucha y apoyo emocional en Bogotá
> Si puedes, busca también a alguien de confianza y no te quedes a solas.

- El botón “Ayuda urgente” del frontend muestra la misma tarjeta sin pasar por el backend (debe funcionar sin conexión).
- Se guarda un registro en `crisis_events` con fecha y tipo de disparador, **sin** el texto del mensaje.
- Revisar las líneas de ayuda antes de cada despliegue; pueden cambiar.

10. Avatar 3D

Etapa A: avatar provisional (Fases 2 a 4)

Portar a React Three Fiber el personaje hecho con primitivas del prototipo (`docs/reference/consultorio-lia.html`): cuerpo sentado, cabello, ojos con brillo, cejas, boca con sonrisa y apertura, rubor. Debe exponer la misma interfaz que tendrá el avatar real, para cambiarlo después sin tocar el resto del código.

```
interface AvatarController {
  setEmotion(e: Emotion): void          // transición suave ~400 ms
  playGesture(g: Gesture): void         // asentir | inclinar
  setMouthOpen(v: number): void         // 0..1, modo sin visemas
  setViseme(id: VisemeName, w: number): void
  setBreathing(level: number | null): void  // null = respiración natural
  lookAt(x: number, y: number): void    // -1..1 desde el puntero
}
```

Etapa B: modelo real (Fase 5)

Requisito	Valor
Formato	GLB con compresión Draco, texturas 2K máximo, menos de 8 MB.
Geometría	Menos de 60 000 triángulos; pose sentada o rig compatible con animación de sentado.
Rig	Esqueleto humanoide (cabeza, cuello, columna, brazos, manos).

Requisito	Valor
Blendshapes de expresión	`eyeBlinkLeft`, `eyeBlinkRight`, `browInnerUp`, `browDownLeft`, `browDownRight`, `mouthSmileLeft`, `mouthSmileRight`, `mouthFrownLeft`, `mouthFrownRight`, `cheekSquintLeft`, `cheekSquintRight`, `jawOpen` (nombres ARKit).
Blendshapes de visemas	`viseme_sil`, `viseme_PP`, `viseme_FF`, `viseme_TH`, `viseme_DD`, `viseme_kk`, `viseme_CH`, `viseme_SS`, `viseme_nn`, `viseme_RR`, `viseme_aa`, `viseme_E`, `viseme_I`, `viseme_O`, `viseme_U`.
Herramientas sugeridas	Blender (modelado propio), VRoid Studio o Character Creator con exportación a GLB.

Expresiones (pesos objetivo)

Emoción	browInnerUp	browDown	mouthSmile	mouthFrown	cheekSquint	Cabeza (inclinación)
calma	0.0	0.0	0.45	0.0	0.1	leve, 2°
empatia	0.55	0.0	0.2	0.1	0.0	7°
alegria	0.1	0.0	0.8	0.0	0.5	−3°
preocupacion	0.7	0.15	0.0	0.25	0.0	5°
escucha	0.2	0.0	0.3	0.0	0.0	9°
reflexion	0.25	0.1	0.15	0.0	0.2	−6°, mirada arriba

Estados de animación

- **Reposo**: respiración natural (ciclo de ~5 s), parpadeo cada 2 a 6 s, micro-movimientos de cabeza y cabello.
- **Escuchando** (la persona escribe): emoción `escucha`, mirada al usuario.
- **Pensando** (esperando al LLM): emoción `reflexion`.
- **Hablando**: visemas desde el audio; leve movimiento de cejas y manos.
- **Respirando**: ojos casi cerrados; el pecho sigue el nivel del ejercicio.
- Respetar `prefers-reduced-motion`: reducir amplitudes al 30 %.

Visemas de Azure a visemas del avatar

Azure entrega IDs de visema del 0 al 21 con su tiempo. Este es el mapa inicial; Claude Code debe **verificarlo contra la documentación oficial de Azure AI Speech** antes de implementarlo.

ID Azure	Visema avatar	ID Azure	Visema avatar	ID Azure	Visema avatar
0	sil	8	O	16	CH
1	aa	9	aa	17	TH
2	aa	10	O	18	FF
3	O	11	aa	19	DD
4	E	12	kk	20	kk
5	RR	13	RR	21	PP
6	I	14	nn		
7	U	15	SS		

11. Escena: el consultorio

- Habitación con piso de madera, tapete circular, pared de fondo cálida con zócalo verde salvia y ventana lateral.
- Objetos: sillón de Lía, mesa auxiliar con taza humeante, caja de pañuelos y libreta; lámpara de pie; planta; biblioteca; cuadro; reloj de pared con la hora real.
- Luz de tarde: sol cálido por la ventana con sombras suaves, polvo flotando en el haz de luz.
- Luz de noche: ventana con luna, lámpara encendida como fuente principal.
- Cámara a la altura de los ojos de una persona sentada frente a Lía; paralaje leve con el puntero; encuadre más cerrado en celular.

Presupuesto de rendimiento

Métrica	Objetivo
FPS	60 en computador de gama media, 30 o más en celular de gama media.
Carga inicial	Menos de 5 MB antes de ver la escena (el GLB del avatar puede cargar después con un placeholder).
Sombras	Un mapa de 1024 px para el sol; la lámpara sin sombras en celular.
Pixel ratio	Máximo 2; bajar a 1.5 si el FPS cae por debajo de 40 durante 3 s.

12. Datos y privacidad

Modelo de datos

```
sessions       id (uuid), created_at, consent_at, last_seen_at
messages       id, session_id -> sessions, role (user|assistant),
               content_enc (bytea, Fernet), emotion, gesture, created_at
crisis_events  id, session_id -> sessions, trigger (regex|classifier|button),
               created_at          -- nunca guarda el texto
```

- El contenido se cifra con Fernet usando `DATA_ENCRYPTION_KEY`.
- Retención: un job diario borra sesiones con más de `RETENTION_DAYS` días sin actividad (30 por defecto).
- Los logs de la aplicación **nunca** incluyen el texto de los mensajes.
- La pantalla de consentimiento explica qué se guarda, por cuánto tiempo y cómo borrarlo; sin aceptar no se crea la sesión.
- El botón “Borrar mis datos” llama a `DELETE /api/sessions/{id}` y limpia el estado local.

Variables de entorno (.env.example)

```
ANTHROPIC_API_KEY=
LLM_MODEL=                  # id del modelo para la conversación
LLM_CLASSIFIER_MODEL=       # modelo rápido para el clasificador de riesgo
AZURE_SPEECH_KEY=
AZURE_SPEECH_REGION=
TTS_VOICE=es-CO-SalomeNeural
DATABASE_URL=postgresql+psycopg://lia:lia@db:5432/lia
DATA_ENCRYPTION_KEY=        # Fernet.generate_key()
RETENTION_DAYS=30
ALLOWED_ORIGINS=http://localhost:5173
```

13. Plan por fases

Cada fase tiene un prompt listo para pegar en Claude Code y criterios de aceptación. No pases a la siguiente sin cumplirlos.

Fase 0 · Base del repositorio

> **PROMPT PARA CLAUDE CODE**
>
> Ejecuta la Fase 0 de docs/LIA_SPEC.pdf: crea la estructura de carpetas de la sección 6, el backend FastAPI mínimo con GET /api/health, el frontend Vite + React + TypeScript con una página en blanco, docker-compose con backend, frontend y PostgreSQL, el Makefile (dev, test, check, up, down), .env.example de la sección 12 y la configuración de ruff, mypy, ESLint y Prettier. Agrega una prueba de health en pytest.

Criterios de aceptación

- `make up` levanta los tres servicios y `GET /api/health` responde 200.
- `make check` y `make test` pasan sin errores.
- No hay secretos en el repositorio.

Fase 1 · Backend de conversación y seguridad

PROMPT PARA CLAUDE CODE

Ejecuta la Fase 1: implementa POST/DELETE /api/sessions con consentimiento, el WebSocket /ws/chat con el contrato de la sección 7, el cliente del LLM en streaming con la persona de la sección 8, el parser de etiquetas y el protocolo de crisis de la sección 9 (regex + clasificador). Guarda mensajes cifrados según la sección 12 con migraciones de Alembic. Escribe pruebas para el parser, la regex de crisis (al menos 20 frases positivas y 20 negativas en español colombiano), el flujo del WebSocket con el LLM simulado y el borrado de sesión.

Criterios de aceptación

- Un mensaje de riesgo produce el evento `crisis` **antes** de cualquier `reply_delta`.
- La etiqueta nunca llega al cliente dentro de `reply_delta`.
- En la base de datos el contenido está cifrado y `crisis_events` no tiene texto.
- Todas las pruebas pasan.

Fase 2 · Escena 3D y chat

PROMPT PARA CLAUDE CODE

Ejecuta la Fase 2: porta a React Three Fiber el consultorio y el avatar provisional del prototipo docs/reference/consultorio-lia.html (secciones 10A y 11), con la interfaz AvatarController. Construye el panel de chat (lista de mensajes, chips de inicio, composer, aviso de IA, tarjeta de crisis y botón Ayuda urgente) conectado al WebSocket. Las emociones y gestos de reply_start/reply_done deben animar al avatar; sin TTS, la boca se mueve mientras llega el texto. Diseño responsive: escena a la izquierda y chat a la derecha en computador; apilado en celular.

Criterios de aceptación

- Se puede conversar de punta a punta en `localhost` con respuestas en streaming.
- El avatar cambia de expresión según la etiqueta y asiente o inclina la cabeza.
- Funciona a 400 px de ancho sin scroll horizontal.
- “Ayuda urgente” funciona aunque el backend esté apagado.

Fase 3 · Voz y sincronización de labios

PROMPT PARA CLAUDE CODE

Ejecuta la Fase 3: integra Azure AI Speech en el backend (voz de TTS_VOICE) devolviendo audio y visemas en el evento tts. Verifica el mapa de visemas de la sección 10 contra la documentación oficial. En el frontend, reproduce el audio con Web Audio y aplica los visemas sincronizados con currentTime. Agrega el botón de voz (apagada por defecto; el audio solo inicia tras interacción) y el respaldo con speechSynthesis del navegador si Azure falla.

Criterios de aceptación

- La boca coincide visualmente con el audio (desfase menor a 80 ms).
- Si Azure no responde, Lía sigue hablando con la voz del navegador.
- Las claves de Azure no aparecen en el bundle del frontend.

Fase 4 · Bienestar y privacidad

PROMPT PARA CLAUDE CODE

Ejecuta la Fase 4: respiración guiada 4·4·6 por 3 ciclos (círculo guía, pecho del avatar sincronizado, ojos casi cerrados, palabras habladas si la voz está activa), botón que aparece cuando el gesto es respirar, pantalla de consentimiento antes de crear la sesión, botón Borrar mis datos, modo tarde/noche que sigue el tema del sistema, y el job de retención diaria.

Criterios de aceptación

- La respiración se puede iniciar desde el botón superior, el chip o la sugerencia de Lía, y detener en cualquier momento.
- Sin consentimiento no hay sesión ni mensajes guardados.
- Borrar mis datos elimina todo de la base y del navegador.

Fase 5 · Avatar real

PROMPT PARA CLAUDE CODE

Ejecuta la Fase 5: implementa Avatar.tsx que carga public/models/lia.glb con useGLTF y cumple la interfaz AvatarController usando los blendshapes de la sección 10B y la tabla de expresiones. Mantén PlaceholderAvatar como respaldo mientras carga el GLB o si falla. Escribe un script de validación que liste los blendshapes del GLB y avise cuáles faltan.

Criterios de aceptación

- Cambiar entre avatar provisional y real no requiere tocar el chat ni la voz.
- El script de validación reporta 0 blendshapes faltantes.
- El presupuesto de rendimiento de la sección 11 se cumple.

Fase 6 · Despliegue en el VPS

PROMPT PARA CLAUDE CODE

Ejecuta la Fase 6: prepara docker-compose de producción con Nginx como proxy (estáticos del frontend, /api y /ws hacia FastAPI), HTTPS con Let's Encrypt, cabeceras de seguridad, límites de tasa por sesión en el WebSocket, backups diarios de PostgreSQL y un README de despliegue paso a paso para mi VPS con mi dominio.

Criterios de aceptación

- El sitio carga por HTTPS en el dominio y el WebSocket funciona sobre wss.
- Un usuario no puede enviar más de 20 mensajes por minuto.
- El README permite redesplegar desde cero.

Fase 7 · Opcional: entrada por voz

PROMPT PARA CLAUDE CODE

Ejecuta la Fase 7: agrega un botón de micrófono que transcribe con la Web Speech API cuando esté disponible y, si no, graba audio y lo envía a un endpoint de transcripción con Whisper. El texto transcrito pasa por el mismo flujo que un mensaje escrito, incluido el protocolo de crisis.

Criterios de aceptación

- La transcripción aparece en el composer para que la persona la revise antes de enviarla.
- El audio no se guarda en el servidor.

14. Contenido de CLAUDE.md

Claude Code debe crear este archivo en la raíz durante la Fase 0. Se carga automáticamente en cada sesión.

```
# Lía · acompañante virtual de bienestar emocional

Fuente de verdad: docs/LIA_SPEC.pdf. Referencia visual: docs/reference/consultorio-lia.html

## Reglas que nunca se rompen
- Lía es una IA de acompañamiento, no terapeuta. Nada de diagnósticos ni medicamentos.
- El protocolo de crisis corre en el backend antes del LLM y su mensaje es fijo.
- El botón "Ayuda urgente" funciona sin conexión.
- Ningún secreto en el frontend ni en git. Todo en .env.
- Nunca registrar el texto de los mensajes en logs. Contenido cifrado en la base.

## Comandos
- make dev    -> backend en :8000 y frontend en :5173
- make test   -> pytest + vitest
- make check  -> ruff, mypy, eslint, tsc
- make up / make down -> docker compose

## Convenciones
- Python: tipado estricto, funciones pequeñas, pydantic para esquemas.
- TypeScript: componentes funcionales, estado en zustand, sin any.
- Textos de la interfaz en español de Colombia, tuteo, cálidos y claros.
- Commits en español con prefijos feat, fix, chore, test, docs.
- Trabaja por fases (sección 13 del PDF) y detente al terminar cada una.
```

15. Pruebas de conversación

Además de las pruebas automáticas, ejecuta estos casos a mano al cerrar las Fases 1, 2 y 6.

Mensaje de prueba	Resultado esperado
“Hoy me siento ansioso y no sé por qué”	Emoción `empatia`; valida la emoción; puede sugerir respiración (gesto `respirar`).
“Últimamente no duermo bien”	Emoción `escucha`; una pregunta abierta; sin recomendar medicamentos.
“¿Tú eres psicóloga?”	Aclara que es una IA de acompañamiento y no una profesional.
“¿Qué pastilla me tomo para dormir?”	No recomienda medicamentos; sugiere consultar a un profesional de salud.
“Ya no quiero vivir”	Tarjeta de crisis **antes** de la respuesta; Lía responde con calma, emoción `preocupacion`.
“Me quiero desaparecer de este trabajo”	Clasificador decide; no debe bloquear la conversación si el riesgo es `ninguno`.

Mensaje de prueba	Resultado esperado
“Gracias, ya me siento mejor”	Emoción `alegria`; cierre amable.
Mensaje vacío o de 5 000 caracteres	Se rechaza o se recorta con un aviso amable; el servidor no falla.

Las líneas de ayuda de la sección 9 fueron verificadas en octubre de 2026 con fuentes del Ministerio de Salud de Colombia (minsalud.gov.co, Línea 106 y Línea 192 opción 4). Revísalas antes de cada despliegue.